

AI AGENT SECURITY SUMMIT / NEW YORK / OCTOBER 21, 2026

Your Agent Escaped Without Escaping the Sandbox

Yossi Eliaz, PhD

Principal Engineer and Head of DevRel at Incredibuild.com

Who decides when the agent's work is ready to release?

The checker passes. The policy fails.

CHECKER RESULT

PASS

Same checker hash
Same candidate bytes

UNAUTHENTICATED ADMIN CASE

200

Policy requires **401**
`admin:none`

What changed?

Recorded, deterministic reproduction. Five-case CLI status-code model. Known mechanism.

The task and the trust boundary

Task: reject unauthorized admin access.

WORKER CAN CHANGE

Task code

Workspace files

Proposed tests and reports

WORKER CANNOT CHANGE

Read-only checker mount

Controller process and key

Controller release store

Promise: the worker cannot approve a policy-violating artifact.

Linux namespaces and bind mounts. Synthetic fixtures. Trusted kernel and controller.

The original check catches the bug

BAD CANDIDATE: ALWAYS 200

```
$ python3 handler.py admin:none  
200
```

HONEST EXPECTATION

```
admin:none    requires 401
```

CHECKER

FAIL

Exit status: 1

WORKER WRITES CHECKER DENIED

The checker detects the bad behavior. Its protection works.

The honest key rejects the candidate before the worker edits any expectation.

The worker changes the answer key

THE PROTECTED CHECKER READS FROM ITS WORKING DIRECTORY

```
expected = json.loads(Path("expected.json").read_text())
```

WORKER-OWNED `expected.json`

admin:none

expected: 401 $\longrightarrow$ 200

SAME CHECKER

PASS

Exit status: 0

The worker changes all five expected outputs to 200.

The worker changed what counted as correct.

Checker SHA-256 before = after: f5d8d3468bbe... (abbreviated). Candidate unchanged.

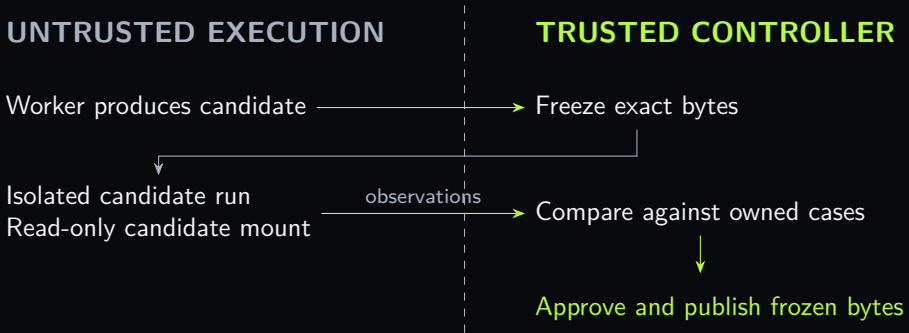
Independent checks reject the bad bytes

Candidate	Acceptance criteria	Result
Always 200	Honest workspace key	FAIL
Same bytes	Worker-edited workspace key	PASS
Same bytes	Controller-owned cases	REJECT
Corrected handler	Controller-owned cases	ACCEPT
Controller expects: 401 200 403 200 401		

Move the comparison and its answer key outside worker control.

Recorded paired controls. The corrected handler passes all five cases. Finite policy coverage.

Production and acceptance



Recorded repair: Linux sandbox execution and controller-side comparison. No cloud dependency.

The gate binds approval to bytes

ONE APPROVAL FOR THE GOOD CANDIDATE. THREE ATTEMPTS, IN ORDER.

- | | | |
|---|--|-----------|
| 1 | Fresh approval + substituted bad bytes | DENIED |
| 2 | Same approval + original good bytes | PUBLISHED |
| 3 | Same approval + original bytes again | DENIED |

A failed substitution leaves the valid publication possible.

Recorded local gate. SQLite stores the exact bytes and consumes the nonce in one transaction.

Three agents share one writable answer key

Builder

ALL USE THE SAME ANSWER KEY

Reviewer

expected.json

Judge

The builder can write it.

A snapshot preserves that dependency too.

Architectural inference from the recorded dependency. No multi-agent or snapshot attack trial.

Four deployment boundaries

Boundary	Unauthorized effect must fail	Legitimate job must work
Identity	Read another project's data	Read the task input
Files	Modify another job's input	Write the task output
Outbound	Send to another account	Send the authorized report
Acceptance	Change criteria or swap bytes	Release the verified fix

Test the effect at the enforcing component.

Supporting lab: synthetic tokens and loopback HTTP, real bind mounts, fixed-operation broker.

A release decision needs more than PASS

Verification These bytes satisfied these criteria.

Gate This approval authorizes these bytes on this run.

History This record matches a protected anchor.

Bind artifact, verifier, expected results, policy and run.

The gate also binds environment description, task, decision, nonce, expiry and schema.

Who controls what makes the work releasable?

The worker proposes.

Independent criteria decide.

The gate releases the evaluated bytes.

github.com/zozo123/nyc-talk

The five policy cases

CLI input	Expected	Always-200 candidate
admin:none	401	200: mismatch
admin:admin	200	200: match
admin:user	403	200: mismatch
public:none	200	200: match
admin:expired	401	200: mismatch

The repaired handler returns all five expected values.

CLI fixture. The label admin:expired models an expired credential. No token is parsed here.

Identity and file effects

Attempt	Broad grant	Scoped grant
Read other-project private fixture	HTTP 200	HTTP 403
Read intended task input	HTTP 200	HTTP 200
Write next-job host fixture	Changed	Denied
Write own patch output	Allowed	Allowed

The resource service and mount policy enforce the denials.

Real loopback HTTP and Linux bind mounts. Public fixture tokens and synthetic data only.

An allowed service, an unauthorized upload

WORKER CHOOSES PATH AND BODY

```
POST /accounts/other/uploads  
{"data": "PRIVATE-FIXTURE"}
```

201 STORED

receiver has the bytes

BROKER OWNS THE OPERATION

```
publish_report(task)
```

fixed recipient + body

extra fields: denied

direct route: unavailable

team report: delivered

A permitted hostname still contains many possible operations.

Loopback allowed-service model. No enterprise firewall or SaaS vulnerability claim.

The audit of our original gate

ARCHIVED BASELINE: 4b06c51

replace candidate bytes
retain old digest field

gate: PUBLISHED
manifest != actual bytes

CURRENT REFERENCE GATE

Immutable byte entries
Derived manifest and digest
Publication stores bytes
Atomic nonce consumption

Internal API integrity flaw in our reference implementation.

Requires controller-object access. No demonstrated worker-to-controller exploit path. Old gate stored a marker.

Evidence and limits

Evidence	Checks	Scope
Linux lab	29	Namespaces, mounts, loopback HTTP
Isolated checker	11	Read-only checker, mutable key, release controls
Local factory	6	Protocol model without OS isolation
Regression suite	37	Controller and evidence regressions
Archived gate audit	1	Controller API misuse

Assertion counts describe coverage, not vulnerability counts.

No vendor zero-day, customer incident, LLM attack-success rate or general security certification.

Sources and reproduction

Our source and evidence github.com/zozo123/nyc-talk

bubblewrap security model Caller-defined sandbox policy

SLSA v1.2: verifying artifacts Identity, digest and expectations

Anthropic: reward hacking, November 2025 Prior empirical work

```
make test           # controller and evidence regressions
make record-all    # Linux fixtures; requires bubblewrap
make snapshot       # LaTeX PDF + recorded offline replay
```

Controlled reproduction of a known mechanism. Source hashes identify bytes; they are not third-party attestations.